CMSC 603 High-Performance Distributed Systems

CUDA asynchronous streams and multi-GPU



Dr. Preetam Ghosh
Professor
Department of Computer Science
pghosh@vcu.edu

Maximizing the occupancy

- Hardware resources available

Nvidia Datacenter GPU	Nvidia A100 SXM	Nvidia H100 SXM	Nvidia H100 PCIe
GPU codename	GA100	GH100	GH100
GPU architecture	Ampere	Hopper	Hopper
GPU board form factor	SXM4	SXM5	PCIe Gen5
Launch date	May 2020	March 2022	March 2022
GPU process	TSMC 7nm N7	custom TSMC 4N	custom TSMC 4N
Die size	826mm2	814 mm2	814 mm2
Transistor Count	54 billion	80 billion	80 billion
FP64 CUDA cores	3,456	8,448	7,296
FP32 CUDA cores	6,912	16,896	14,592
Tensor Cores	432	528	456
Streaming Multiprocessors	108	132	114
Peak FP64	9.7 teraflops	30 teraflops	24 teraflops
Peak FP64 Tensor Core	19.5 teraflops	60 teraflops	48 teraflops
Peak FP32	19.5 teraflops	60 teraflops	48 teraflops
Peak FP32 Tensor Core	156 teraflops | 312 teraflops*	500 teraflops | 1,000 teraflops*	400 teraflops | 800 teraflops*
Peak BFLOAT16 Tensor Core	312 teraflops | 624 teraflops*	1,000 teraflops | 2,000 teraflops*	800 teraflops | 1,600 teraflops*
Peak FP16 Tensor Core	312 teraflops | 624 teraflops*	1,000 teraflops | 2,000 teraflops*	800 teraflops | 1,600 teraflops*
Peak FP8 Tensor Core	-	2,000 teraflops | 4,000 teraflops*	1,600 teraflops | 3,200 teraflops*
Peak INT8 Tensor Core	624 TOPS | 1,248 TOPS*	2,000 TOPS | 4,000 TOPS*	1,600 TOPS | 3,200 TOPS*
Peak INT4 Tensor Core	1,248 TOPS | 2,496 TOPS*	-	-
Interconnect	NVLink: 600GB/s PCI Gen4: 64GB/s	NVLink: 900GB/s PCI Gen5: 128GB/s	NVLink: 600GB/s PCI Gen5: 128GB/s
Max TDP	400 watts	700 watts	350 watts

*Effective TFLOPS or FLOPS using the Sparsity feature

Maximizing the occupancy

- Warp schedulers find a warp that is ready to execute its next instruction, available execution cores, and then start execution

Limitations depend on Compute Capability, e.g. 9.0 (H100):

- max threads / block = 1024 (32 warps)
- max threads / SM = 2048 (64 warps)
- 32 bit registers / SM = 64k
- max shared memory / SM = 64KB
- The number of blocks that run concurrently on a SM depend on the resources requirements of the block (threads, registers, shared memory)
- Minimize register and shared memory, use optimal number of threads per block

Maximizing the occupancy

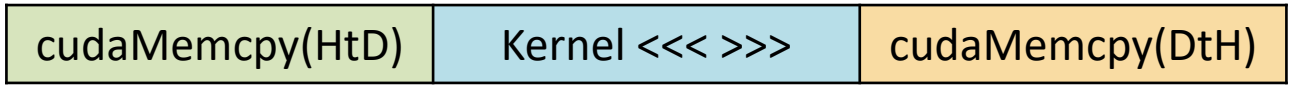
- **occupancy** = fitting warps per SM / max warps per SM (effective usage)
- max warps per SM depends only on GPU
- Fitting warps depends on blocks/registers/shared memory per block

100% occupancy	50% occupancy
2 blocks of 1024 threads	1 block of 1024 threads
32 registers/thread	64 registers/thread
24KB of shared memory / block	48KB of shared memory / block

- Use the CUDA occupancy calculator to maximize the occupancy!
- See Compute Capability https://developer.nvidia.com/cuda-gpus

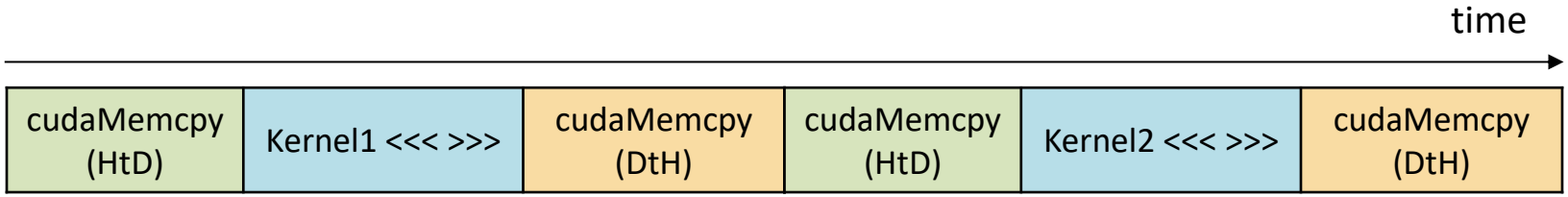
GPU processing workflow

1. Copy input data from CPU memory to GPU memory
2. Launch a GPU kernel
3. Copy results from GPU memory to CPU memory



- Kernel needs to wait input data to be transferred
- Results cannot be copied back until kernel finished
- What if we split and launch multiple kernels?
- Need to wait until **all** input data is copied to start the kernel?
- Need to wait until **all** kernel finish to start copying results?

Execution of multiple kernels with **no** dependencies (decomposable kernel)

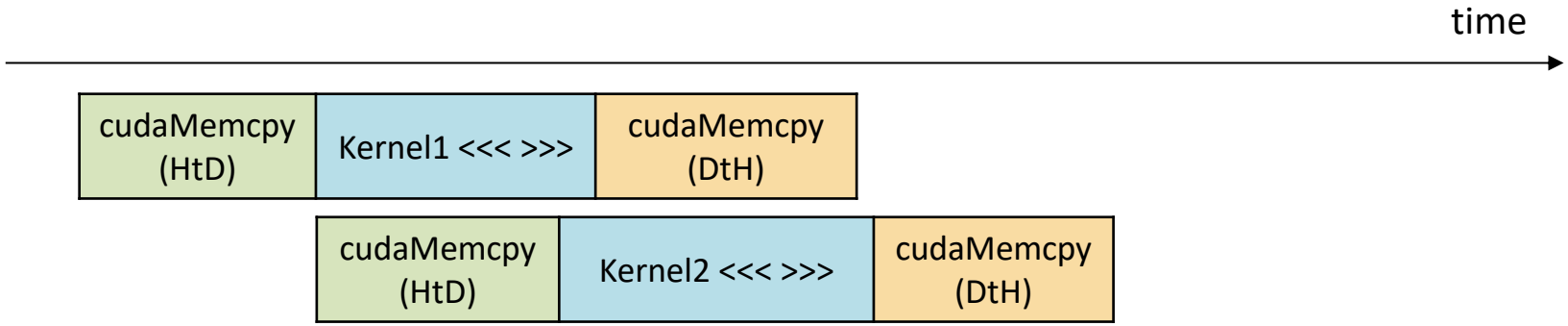



```
cudaMemcpy(d_input1, h_input1, size, cudaMemcpyHostToDevice);
Kernel1<<<blocks1, threads1>>>(d_input1, d_output1);
cudaMemcpy(h_output1, d_output1, size, cudaMemcpyDeviceToHost);

cudaMemcpy(d_input2, h_input2, size, cudaMemcpyHostToDevice);
Kernel2<<<blocks2, threads2>>>(d_input2, d_output2);
cudaMemcpy(h_output2, d_output2, size, cudaMemcpyDeviceToHost);
```

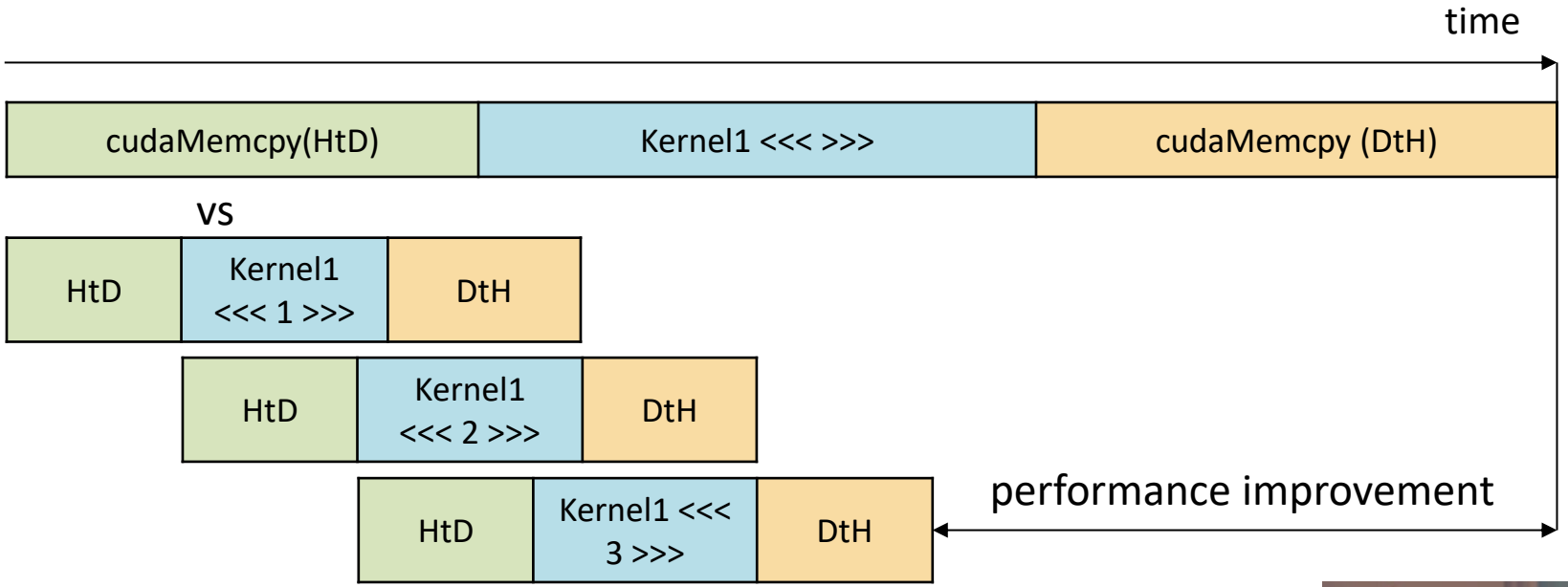
- Assuming Kernel1 and Kernel2 are data-independent
- Is it necessary to serialize the kernel executions? Nope
- Is it necessary to serialize the data transfers? Nope
- Can we overlap them? Yes

Pipelining using **streams**



- Independent data transfer and kernel execution flows
- Maximize the **occupancy** of the GPU resources
- Minimize the **latency** of the program
- Overlapping of data transfer and execution using CUDA **streams**
- Streams simulate multiple pipelines

Splitting a large kernel into smaller chunks



- Divide into smaller chunks
- Multiple concurrent streams to process each chunk
- Overlapping of data transfer (HtD and DtH) and execution



CUDA streams

- A stream is a queue of device work
- The host places work in the queue and continues on immediately
- The device schedules work from streams when resources are free
- Operations **within** a stream are ordered (FIFO) and cannot overlap
- Operations in different streams are unordered and can overlap

Declaration of streams

```
cudaStream_t *streams = (cudaStream_t*)malloc(n * sizeof(cudaStream_t));

for(int i = 0; i < n; i++)
        cudaStreamCreate(&streams[i]);
…

for(int i = 0; i < n; i++)
        cudaStreamDestroy(&streams[i]);
```

Scheduling a kernel to execute in a stream

- Kernels parameters:
  - Blocks setup (1D, 2D, 3D blocks)
  - Threads setup (1D, 2D, 3D threads)
  - Shared memory amount
  - Stream

Kernel <<< blocks, threads, smem, stream >>> ();

Default stream

- Default stream used when no stream is specified, referred as 0
- Completely synchronous w.r.t. host and device

Kernel concurrency

- Assume a *kernel* only utilizes 50% of the GPU
- Default stream

```
kernel <<<blocks, threads>>>();
kernel <<<blocks, threads>>>();
```



- Default stream + user streams

```
cudaStream_t stream1;
cudaStreamCreate(&stream1);
kernel <<<blocks, threads>>>();
kernel <<<blocks, threads, 0, stream1>>>();
cudaStreamDestroy(stream1);
```

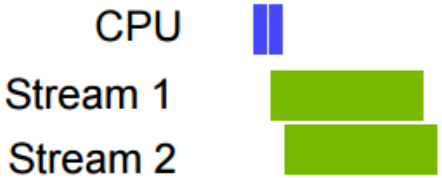


Kernel concurrency

- Assume *kernel* only utilizes 50% of the GPU. Then there are resources to fully host two concurrent kernel executions

- User streams

```
cudaStream_t stream1, stream2;
cudaStreamCreate(&stream1);
cudaStreamCreate(&stream2);
kernel <<<blocks, threads, 0, stream1>>>();
kernel <<<blocks, threads, 0, stream2>>>();
cudaStreamDestroy(stream1);
cudaStreamDestroy(stream2);
```



Requirements for kernel concurrency

- CUDA operations must be in different streams
- Sufficient resources must be available (see CUDA Occupancy calculator)
  - Device resources (multiprocessors, registers, blocks, shared memory, etc)

Requirements for asynchronous memory transfers

- *cudaMemcpyAsync* with host from pinned/page-locked memory
  - Allocate with *cudaMallocHost()* or *cudaHostAlloc()*
  - Release with *cudaFreeHost()*
- *cudaMemcpyAsyncs* in different directions (HtD, DtH)
- Careful *cudaMemcpyAsync* is **non-blocking** in contrast to default *cudaMemcpy*!

Kernel + asynchronous memory transfers

- Transfers must be in a **non-default** stream
- Must use *cudaMemcpyAsync*
- 1 transfer per direction at a time, i.e. multiple transfer on the same direction cannot overlap and will be serialized

```
cudaMallocHost(&h_ptr, bytes);
cudaMalloc(&d_ptr, bytes);
cudaMemcpyAsync(d_ptr,h_ptr,bytes,cudaMemcpyHostToDevice, stream);
kernel << blocks, threads, smem, stream >>> (d_ptr);
cudaMemcpyAsync(h_ptr,d_ptr,bytes,cudaMemcpyDeviceToHost, stream);
```

Concurrency examples

- Synchronous

  cudaMemcpy(...);

  kernel <<<...>>>();



- Asynchronous single stream

  cudaMemcpyAsync(...,stream1);

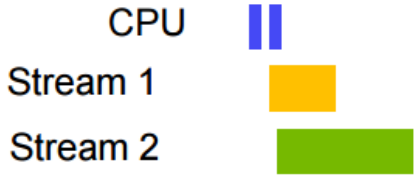
  kernel <<<...,stream1>>>();



- Asynchronous multiple streams

  cudaMemcpyAsync(...,stream1);

  kernel <<<...,stream2>>>();



Synchronization

- Synchronize everything

  cudaDeviceSynchronize()

  - Blocks until all issued CUDA calls are complete (all streams)

- Synchronize host w.r.t. a specific stream

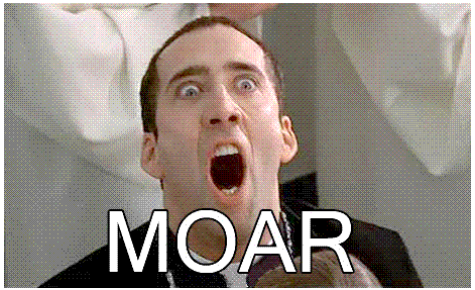
  cudaStreamSynchronize(stream)

  - Blocks until all issued CUDA calls in stream are complete

- Synchronize host or devices using events

  - We already used events to measure elapsed time

Why multi-GPU?



- Further speedup computation
- Data exceeds single-GPU memory capacity
- Multiple GPUs per node improves perf./watt

Inter GPU-communication

- GPUs in the same node can communicate via P2P addressing (GPU to GPU) or shared host memory (GPU to host to GPU)
- GPUs in different nodes communicate via host-side message passing (MPI) (PCIe v3 x16 ~32 GB/s)
- Recently, NVIDIA NVLink provides P2P communication between a GPU and another GPU at a rate up to 300 GB/s.

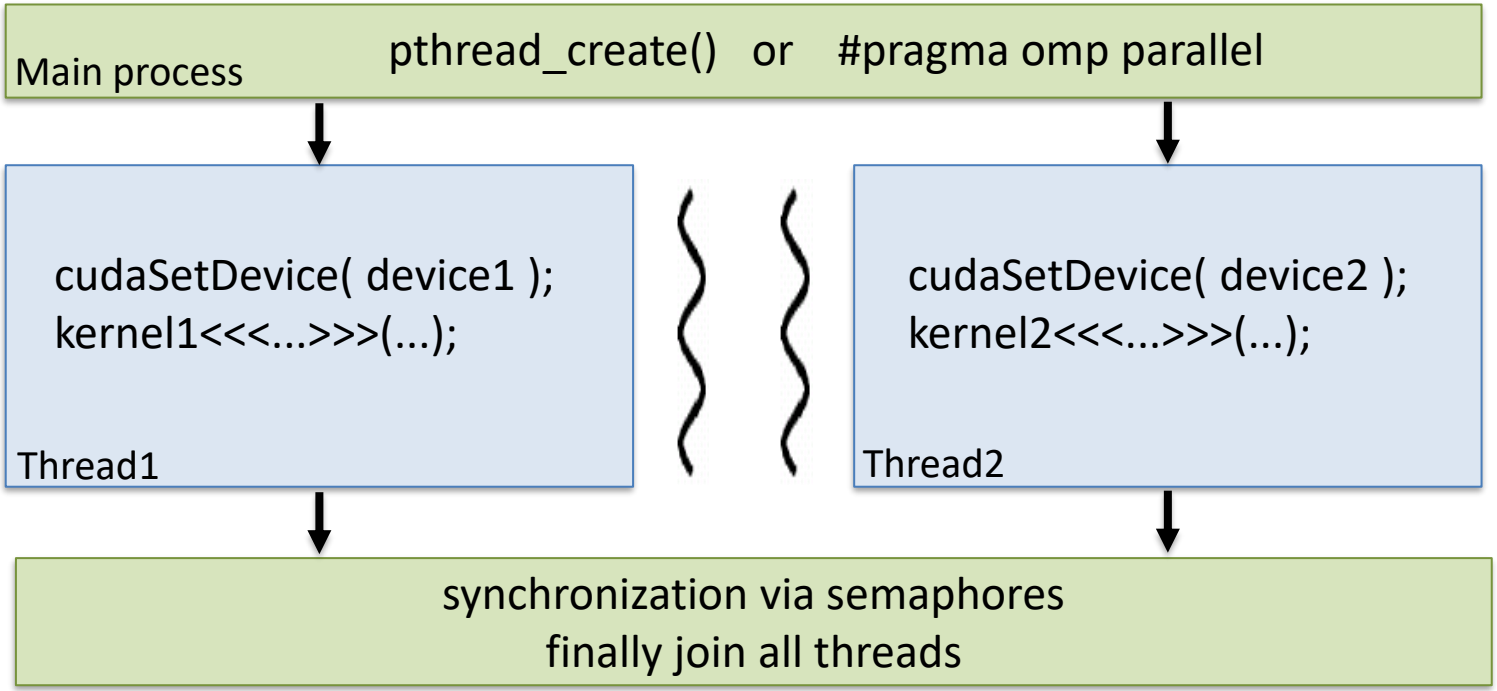
Managing multiple GPUs in a node from a **single** CPU thread

- CUDA call are issued to the current GPU (default GPU 0)
- *cudaSetDevice()* sets the current GPU
- *cudaGetNumDevices(&numDevices)* gets the number of GPUs
- GPU can be changed while async calls (kernels, memcopy) run on other GPUs:
  - Queue up a bunch of async calls to a GPU and then switch to another GPU
  - The following code will have both GPUs executing concurrently
  - A single CPU thread controls multiple GPUs
  - Async and kernel functions are non-blocking

```
cudaSetDevice( 0 );
cudaMemcpyAsync(...);
kernel1<<<...>>>(...);
cudaMemcpyAsync(...);
cudaSetDevice( 1 );
kernel2<<<...>>>(...);
```

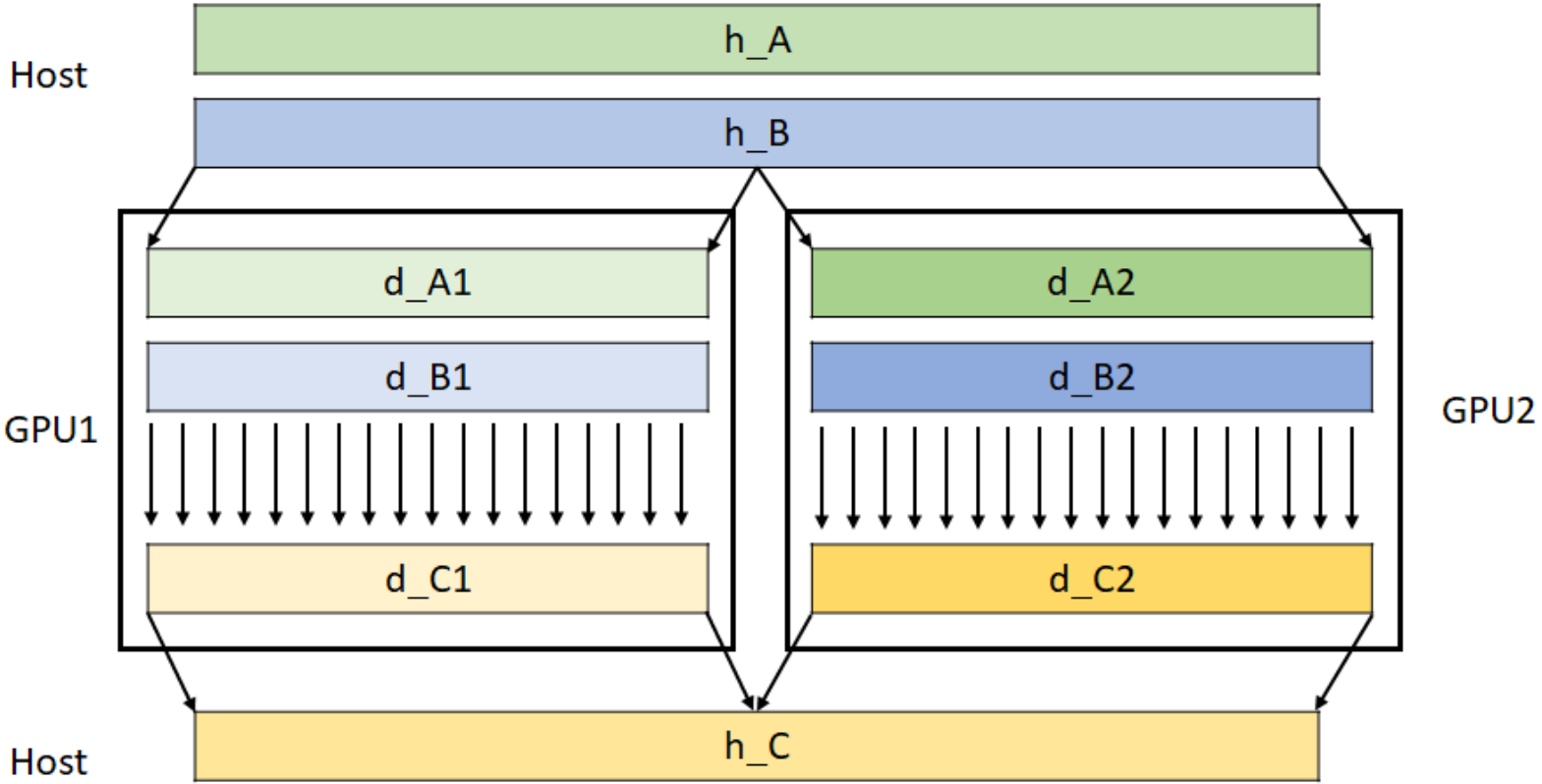
Managing multiple GPUs in a node from **multiple** CPU threads

- Best to have explicit control on the synchronization threads - GPU

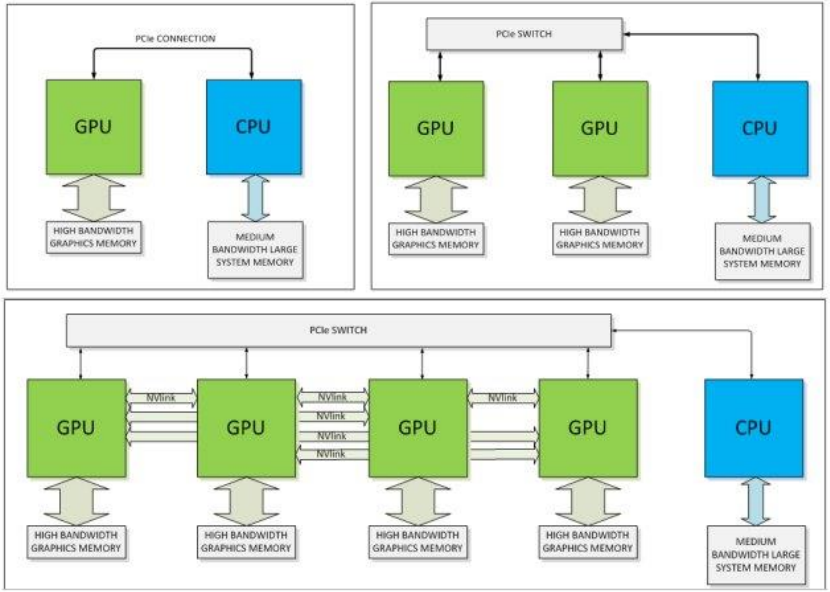


Example: vectorAdd using one host with two GPUs

Example: vectorAdd_multigpu.cu



Peer-to-peer memory access



- Allows data allocated in one GPU to be accessed from a kernel executing in other GPU
- Best to combine memory capacities if needed
- **Very long** latency and small bandwidth!!!!

```
cudaSetDevice( 0 );
…
cudaMalloc(d_ptr, size);
…
cudaSetDevice( 1 );
…
kernel<<<...>>>(d_ptr);
```

Peer-to-peer memory copy

- *cudaMemcpyPeerAsync(void* dst_addr, int dst_dev,*
  void src_addr, int src_dev,*
  size_t num_bytes, cudaStream_t stream)
- Copies amount of bytes between two devices
- Performance is maximized when stream belongs to the source GPU
- There is also a blocking (as opposed to async) version

If peer-access is enabled:

- Bytes are transferred along the shortest PCIe/NVLINK path
- No staging through CPU memory (cool!)

If peer-access is not available:

- CUDA driver stages the transfer via CPU memory ☹

Communication between GPUs in **different** nodes in a network

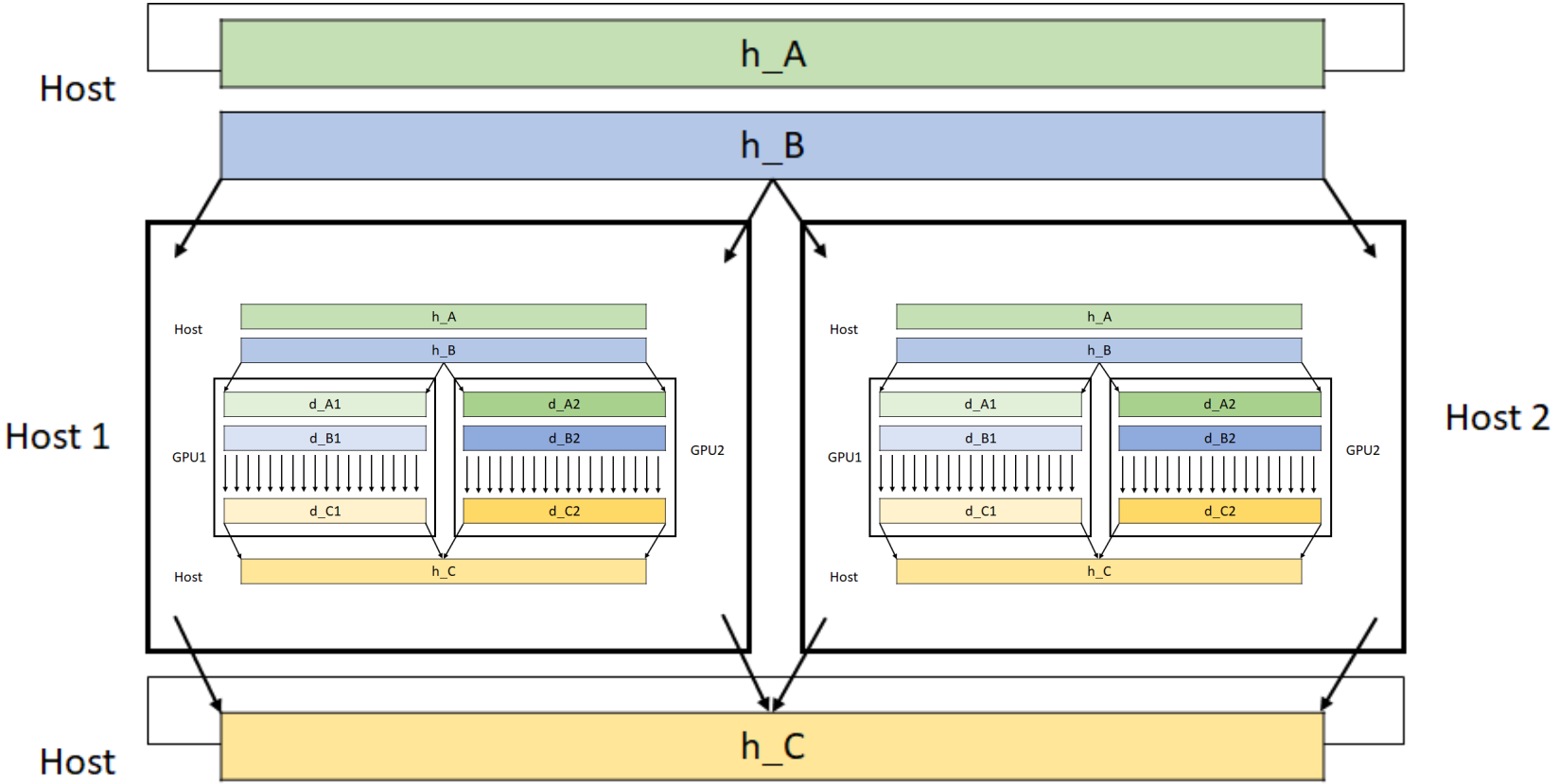
- Requires low-latency high-bandwidth network communication
- Steps for a traditional data exchange:
  1. GPU1->CPU1 transfer (PCIe limited)
  2. CPU1 exchanges via network with CPU2 (MPI?) (network limited)
  3. CPU2->GPU2 transfer (PCIe limited)
- If each node also has multiple GPUs:
  - Can continue using P2P within the node
  - Can overlap some PCIe transfers with network communication
  - In addition to concurrent kernel execution
- GPUDirect for GPU to GPU transfers in an InfiniBand network (DGX H100)

Using MPI for nodes communication

- Two-level parallelization/synchronization (inter vs intra node)
- Computational task is divided into many nodes
- Workload within a node is distributed among its GPUs
- Synchronization among the streams in a node
- Synchronization among the nodes in the cluster
- Use asynchronous non-blocking instructions when possible
- Code pattern:

```
cudaMemcpyAsync( ..., stream[i] );
kernel <<<  … , stream[i] >>> ( ... );
cudaStreamSynchronize( stream[i] );
MPI_Send/recv( ... );
cudaMemcpyAsync( ..., stream[i] );
```

Example: vectorAdd using two hosts with two GPUs



CMSC 603 High-Performance Distributed Systems

CUDA asynchronous streams and multi-GPU



Dr. Preetam Ghosh
Professor
Department of Computer Science
pghosh@vcu.edu